

GAMEPLATFORM

遊戲對戰平台

從會員登入、遊戲大廳、組隊與房間，
到進入遊戲的一體化平台

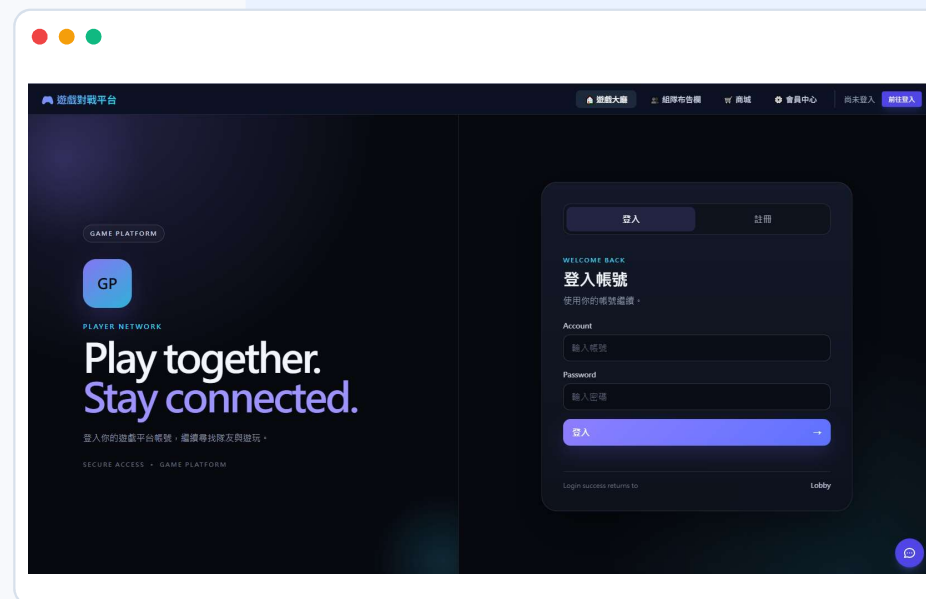
Java 全端工程師職訓・小組專題成果報告

Spring Boot

SQLite

REST API

WebSocket



我們做的不是單一遊戲，而是一條完整的遊玩流程

玩家真正需要的，是從登入、找到遊戲，到和其他玩家一起開始遊戲都能順暢銜接。



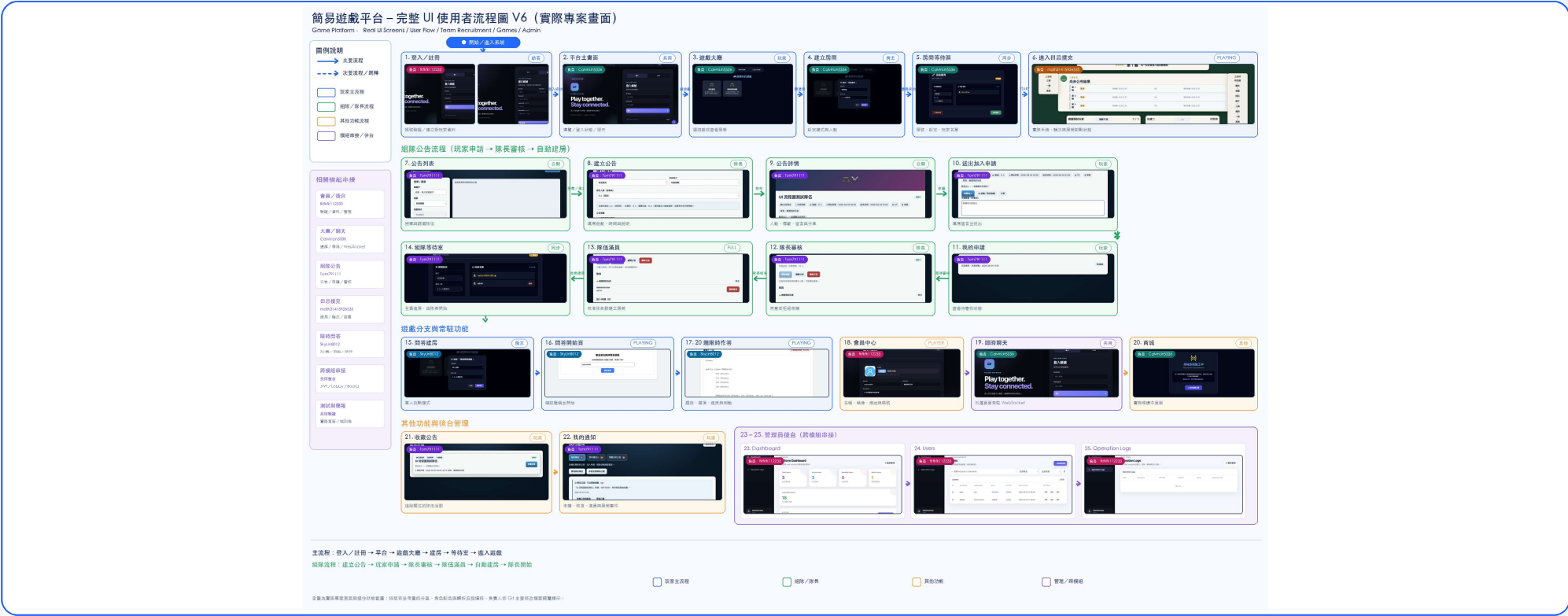
共用會員、遊戲目錄與房間流程，個別遊戲只專注自己的規則與對局。

六個模組共同組成遊戲平台

這一頁只看「系統責任」，不是組員分工。



完整 使用者流程圖：實際畫面串接



主流程

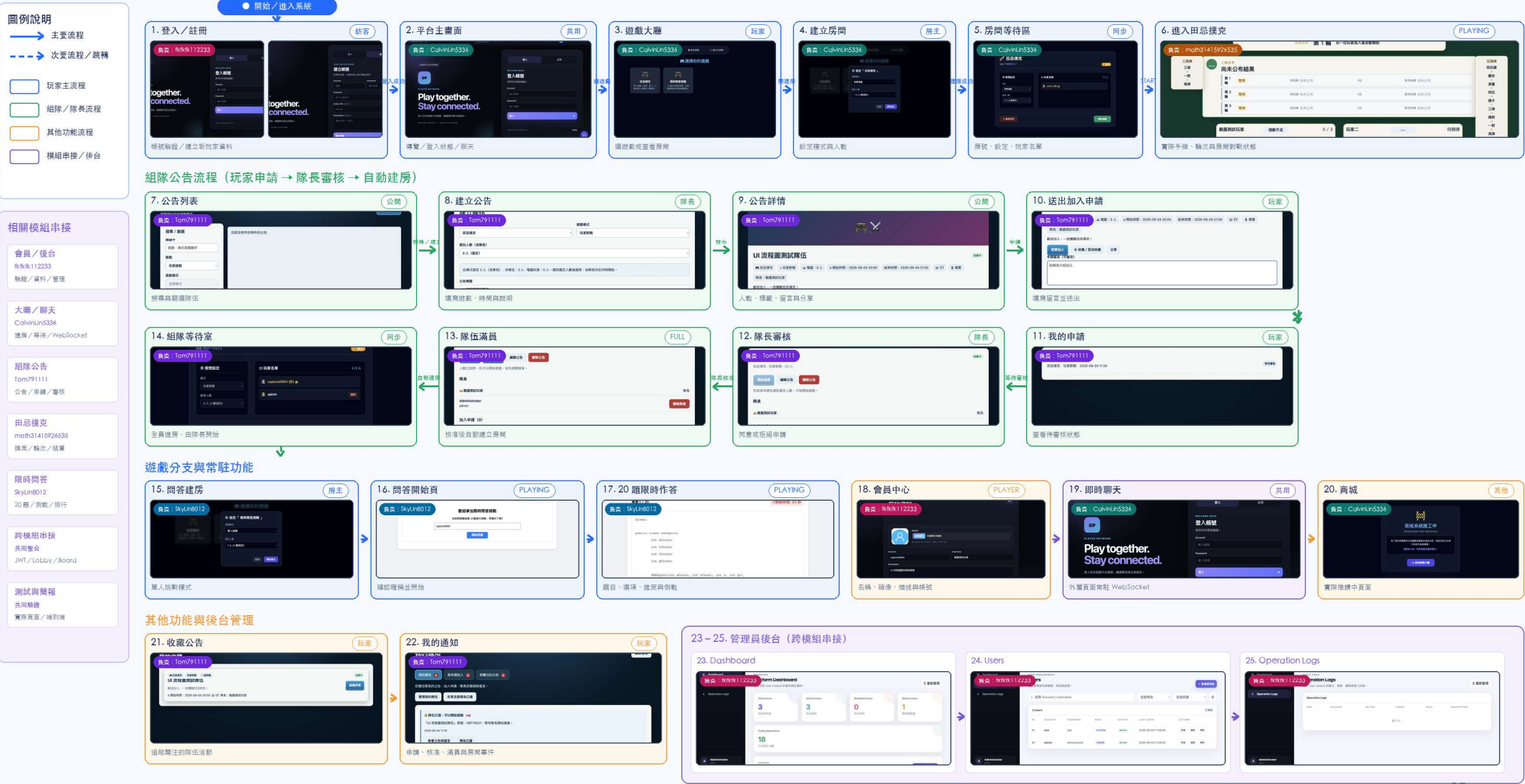
登入 / 註冊 → 平台主頁 → 遊戲大廳 → 建立房間 → 等待室 → 進入遊戲

組隊流程

公告 → 申請加入 → 隊長審核 → 隊伍額滿 → 自動建房

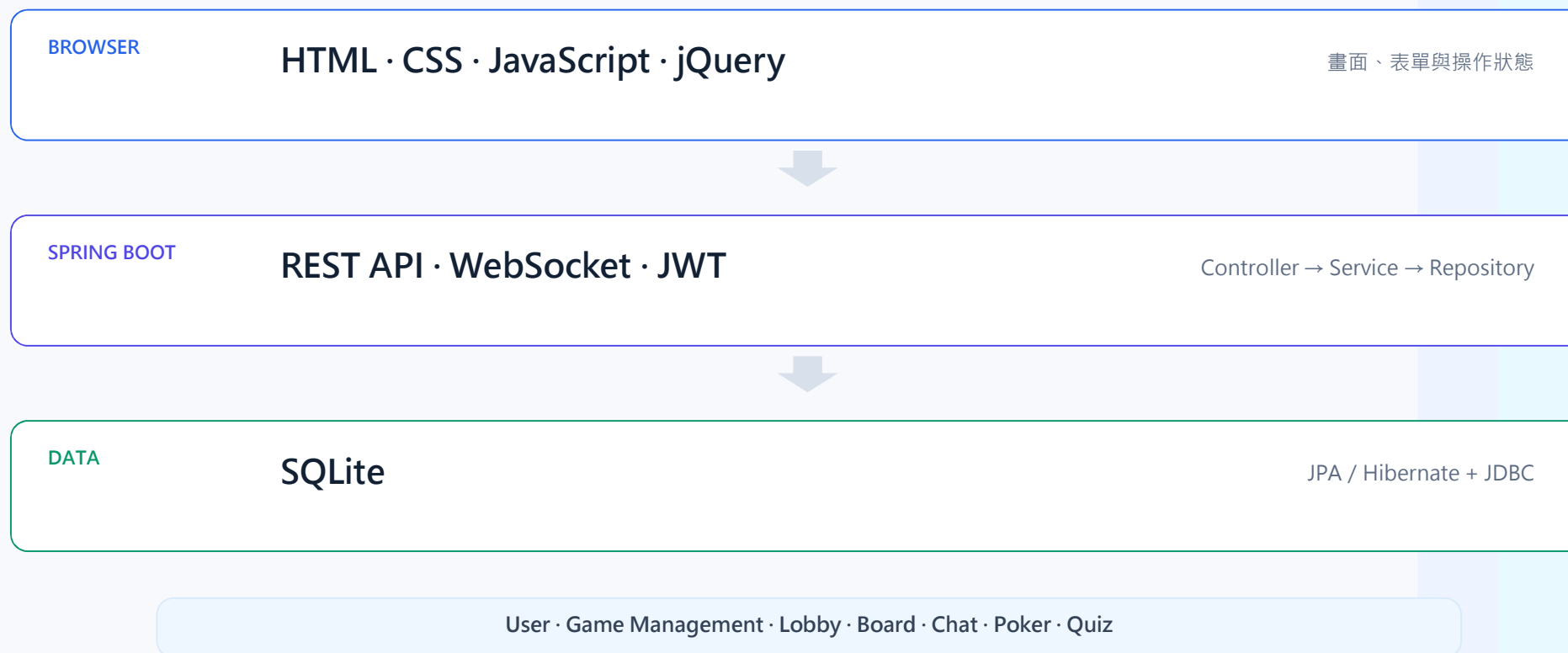
簡易遊戲平台 – 完整 UI 使用者流程圖 V6 (實際專案畫面)

Game Platform - Real UI Screens / User Flow / Team Recruitment / Games / Admin



前端、Spring Boot 與 SQLite 串起所有模組

主要採用 HTML、CSS、JavaScript 與 jQuery，後端由 Spring Boot 提供 API。



網頁使用流程：從登入到進入遊戲

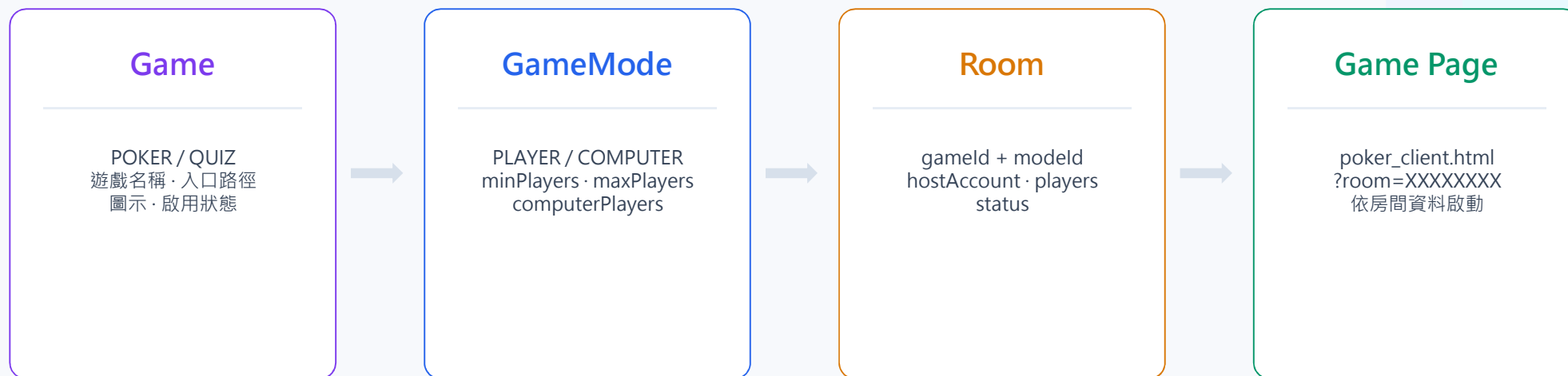
同一個平台身分貫穿所有步驟，遊戲頁不需要再次建立獨立帳號。



從資料庫讀取遊戲路徑 => 決定要開啟哪個遊戲頁面 => room ID 讓遊戲取得本次房間、模式與玩家資訊。

Game、GameMode 與 Room 把遊戲設定從房間邏輯中抽離

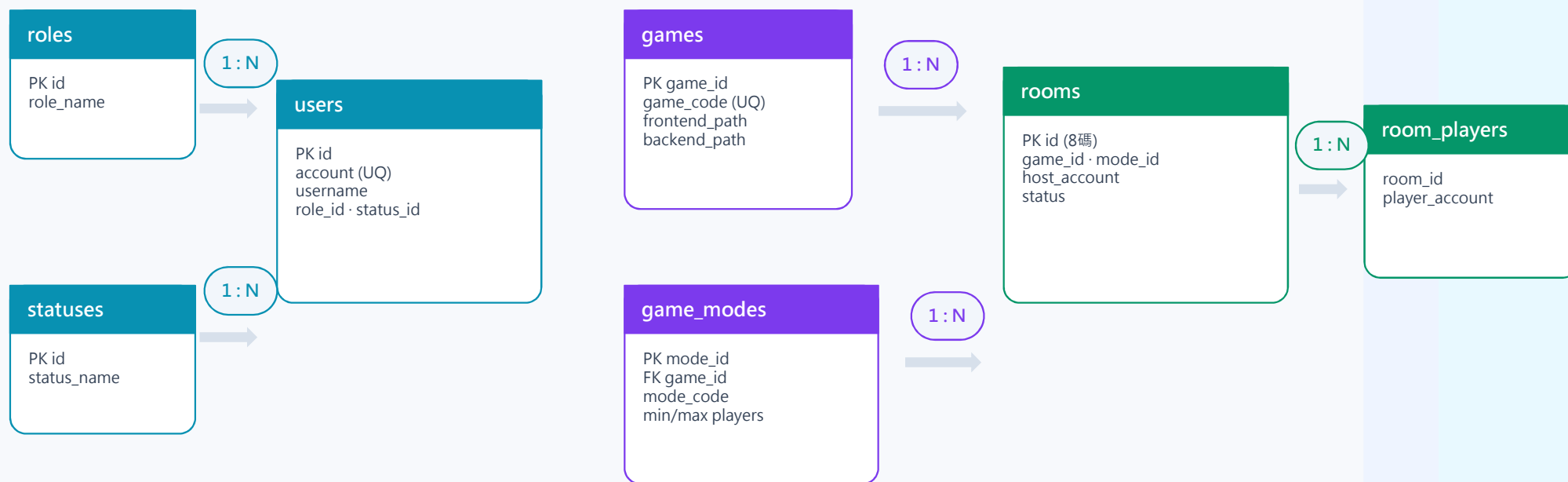
Game 保存遊戲入口，GameMode 保存模式限制，Room 保存一次等待中的對局。



房間只依 gameId / modeId 查規則，不需要把 Poker 或 Quiz 的特殊玩法寫死在通用房間模組。

核心 ER Model：會員、遊戲目錄與房間

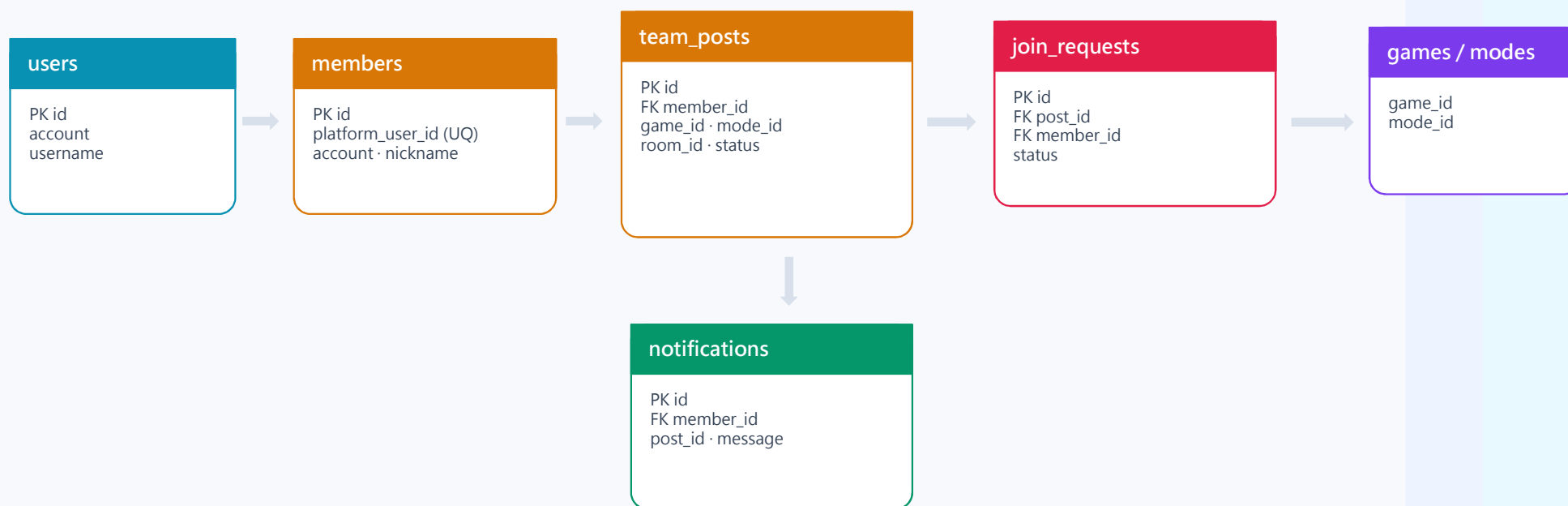
先看會直接影響「登入 → 選遊戲 → 建房 → 進遊戲」的資料表。



rooms 同時保存 game_id 與 mode_id ; room_players 是 Room.players 的 ElementCollection 關聯表。

延伸 ER Model：組隊公告如何連到會員、遊戲與房間

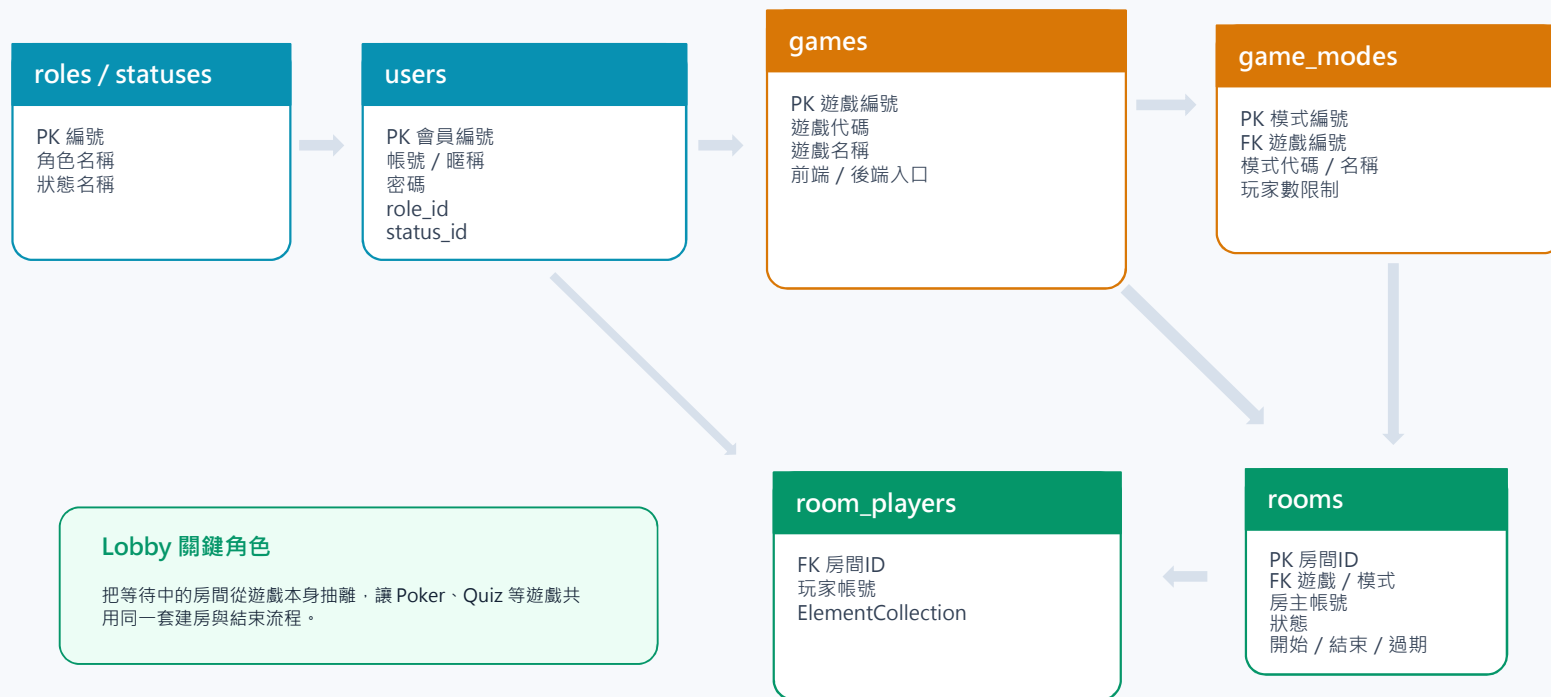
Board 有自己的 Entity，但透過 platform_user_id、game_id、mode_id、room_id 與其他模組銜接。



重點：組隊模組不直接取代會員 / 房間資料表，而是保存必要的識別資訊與自己的流程狀態。

ER Model · Core / Lobby : 會員、遊戲目錄與房間生命週期

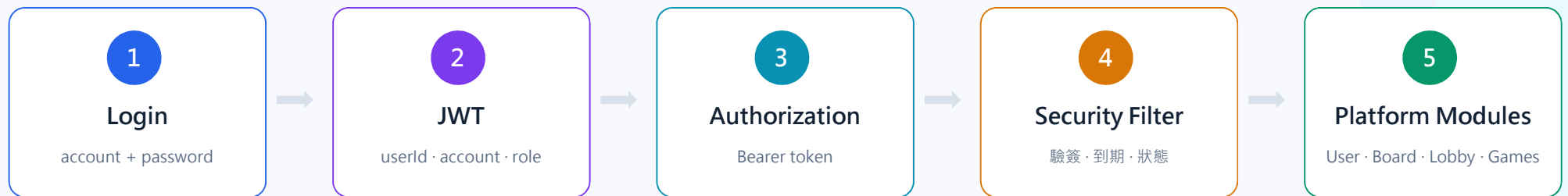
Lobby 是遊戲大廳與房間系統，負責建房、等待、開始遊戲，以及遊戲結束後的房間狀態收斂。



AUTHENTICATION FLOW

JWT 讓不同頁面共享同一個登入身分

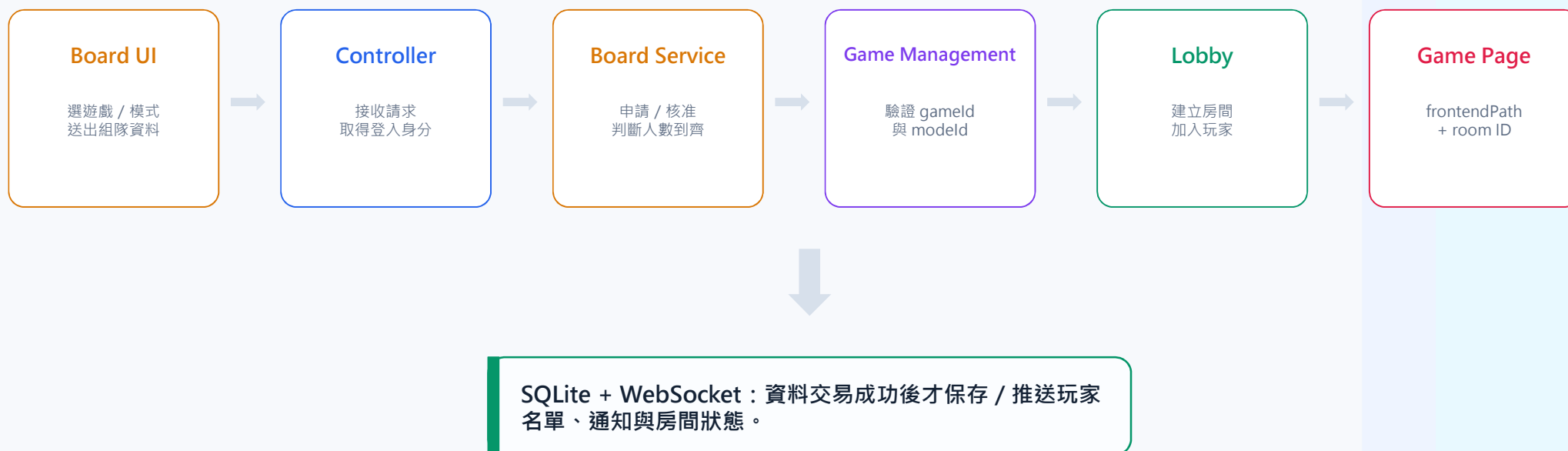
前端保存登入結果；每次受保護請求仍由後端重新驗證 token 與帳號狀態。



localStorage 只保存資料；真正能不能操作，仍以後端驗證結果為準。

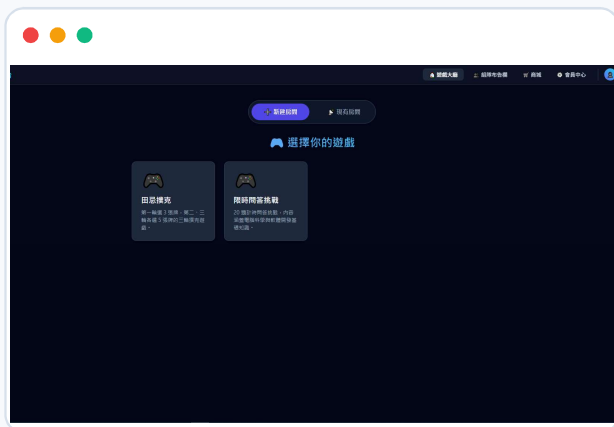
核心功能資料流：組隊完成後建立房間並導向遊戲

用一次核心流程說明前端、Controller、Service、Game Management、Lobby 與 DB 如何串接。



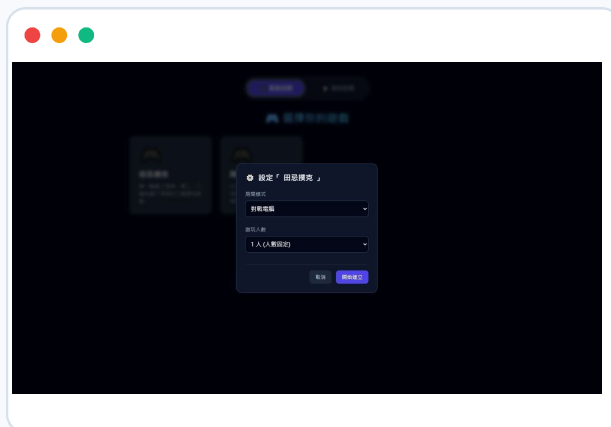
實際畫面把平台流程連成一致體驗

遊戲大廳 → 組隊設定 → 房間等待，畫面都使用同一個登入身分。



01 選擇遊戲

遊戲清單由 Game Management 提供。



02 設定隊伍

模式與人數依資料動態產生。



03 等待開始

房號、房主與玩家名單同步顯示。

田忌撲克完成選牌、比牌與三輪結果呈現

同一頁完成玩家資訊、目前輪次、選牌區、結果區與三輪戰績呈現。

自動選牌



房間資訊 + 玩家身分 + 手牌 + 本輪結果

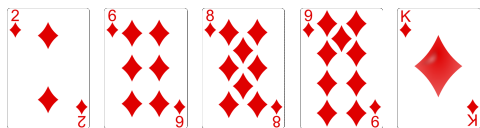
三輪結果



同一頁顯示三輪戰績與對局結果

Poker 演算法：把牌型轉成可直接比較的 strength

以 ♦2、♦6、♦8、♦9、♦K 同花為例，將牌型、點數與花色編碼成單一 long 值。



513090806021 = 5 | 13 | 09 | 08 | 06 | 02 | 1

牌型 同花 最大K 9 8 6 2 花色♦

範例：♦2、♦6、♦8、♦9、♦K = 同花

強度編碼

牌型放在最高位
點數由大到小排列
kicker / 花色放在低位
比較時只看 long 大小

規則集中

牌型規則集中維護
牌組統一轉成 strength
避免大量 if / else

自動選牌基礎

find_best_cards() 列舉組合
每組計算 strength
保留最高分牌組

自動選牌：find_best_cards() 找出本輪最強手牌

列舉本輪可用牌的候選組合，將每一組轉成 strength，比較後保留分數最高的牌組。



find_best_cards() 核心概念

```
best_score = -1
for each combination C(n, k):
    score = score(current, count)
    if score > best_score:
        best_score = score
        best = current
```

回合策略

依目前流程先選第 3 輪 5 張，再選第 2 輪，最後處理第 1 輪 3 張。

目前輪次 自動選牌

auto_choose_current_round() 只重算目前輪次；已完成輪次保持鎖定。

與 strength 的關係

每一組候選牌都交給 score()，再由 Round.determine_hand_strength() 取得分數。

重點：比牌規則只維護一份，自動選牌只需要反覆比較 strength。

限時問答挑戰：可以挑戰、並提供檢討

現時30秒答題挑戰，考驗玩家對JAVA程式基本概念的了解，並提供閱讀與維護的功能

亂數排列產生答題



玩家身分 加入挑戰，並顯示剩餘時間

得分與閱卷檢討



同一頁挑戰分數，並提供答案對比的檢討結果

限時問答挑戰：題目維護與排行榜

提供題目新增、修改、刪除等維護功能，並將玩家成績做成排行榜以資鼓勵

題目維護頁面

刪除

+ 新增題目

儲存題目 取消 / 重設

📖 現有題庫列表

ID	題目內容	限時	選項數	操作
1	How many of these compile? 18:Comparator<String> c1=(j,k)->0; 19:Comparator<String> c2=(String j,String k)->0; 20:Comparator<String> c3=(var j,String k)->0; 21:Comparator<String> c4=(var j,k)->0; 22:Comparator<String> c5=(var j, var k)->0;	30 秒	6 個	編輯 刪除

玩家登入即可維護問答题目

玩家分數排行榜

問答挑戰(20題)

完整閱讀區

🔍 題目管理維護

🏆 排行榜

🏆 玩家最高分排行榜(Top 10)

名次	玩家名稱	最高得分
1	PlayerOne	100
2	QuizMaster	90
3	LuckyGuy	75
4	admin	70
5	root	65

會記錄玩家分數，依照高低排行成績

從會員到遊戲，平台已形成可展示的完整流程

GamePlatform

共用登入、遊戲目錄、組隊與房間流程，再由個別遊戲處理自己的規則。

端到端

登入 → 房間 → 遊戲

模組化

共同平台 + 獨立遊戲

可驗證

58 項測試 + 實機流程

DEMO · Q&A